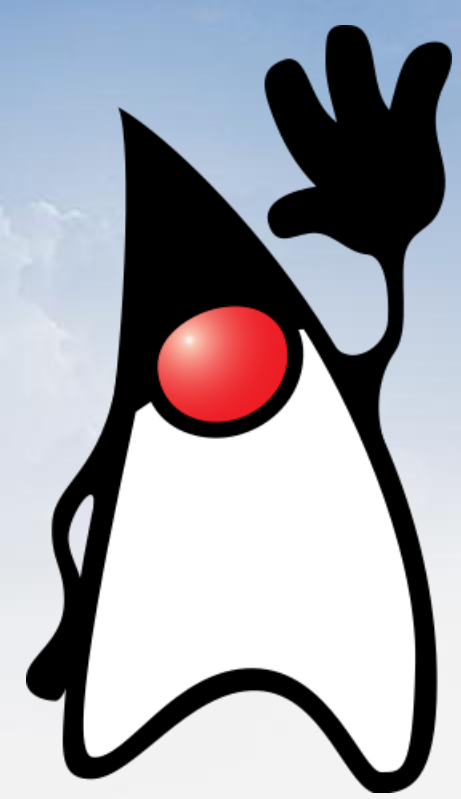




ArrayList<T >

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

Array Quick Review

- We have seen how arrays in Java are implemented.
- However the arrays we have been using up to this point are not dynamic.
 - Cannot change in size during the program run
- Collection classes allows us create an array that can be changed dynamically

ArrayList in Java

- `import java.util.ArrayList;`
- ArrayList can only store one type of data, like arrays
- The ArrayList class simulates the dynamic array component
 - Where the size can change during the program's run
- Lots of operations in the ArrayLists can be executed in $O(1)$ time.
- Expanding the ArrayLists can be executed in $O(n)$ time.
- Search is $O(n)$

Methods of the ArrayList Class

- add
- clear
- contains
- get
- indexOf
- remove
- size

Creating an ArrayList

- When we create an arrayList it has 10 positions
- An overloaded constructor exists if more elements are needed.
 - Why would it be a bad idea to create an ArrayList Object with 10000000 elements?
- We need to state the data type inside the ArrayList
- It is better you don't touch the initial capacity much.

Creating an ArrayList

```
ArrayList<Integer> arr;
```

```
arr=new ArrayList<>();
```

```
//Another syntax
```

```
ArrayList<Integer> arr = new ArrayList<>();
```

```
ArrayList<Integer> arr = new ArrayList<Integer>(10);
```

```
//initial capacity as 10
```

```
ArrayList<Long> arr;
```

```
arr=new ArrayList<>();
```

```
//Another syntax
```

```
ArrayList<String> arr1 = new ArrayList<>();
```

```
//ArrayList<String> arr1 = new ArrayList<String>(); it is also ok
```

```
ArrayList<Double> arr2 = new ArrayList<>(10);
```

```
//initial capacity as 10
```

add(Integer i) and get(int i)

- Adds an element to the end of the ArrayList or at a specific index in the ArrayList

```
ArrayList<Integer> arr=new ArrayList<>();
```

```
int n=5;  
for (int i = 1; i <= n; i++) {  
    arr.add(i);  
}  
  
for (int i = 0; i < arr.size(); i++) {  
    System.out.println(arr.get(i));  
}
```

add(String i) and get(int i)

- Adds an element to the end of the ArrayList or at a specific index in the ArrayList

```
ArrayList<String> arr=new ArrayList<>();

int n=5;
for (int i = 1; i <= n; i++) {
    arr.add("Number"+i);// created a String
}

for (int i = 0; i < arr.size(); i++) {
    System.out.println(arr.get(i));
}
```

ArrayList expansion

- size() is the number of elements inside the ArrayList
- Capacity is the number of available slots in the ArrayList
- When we add element to the ArrayList
 - if the capacity is full, it will get doubled,
 - Size will increase by one
- When we remove an element from the ArrayList
 - Size will decrease by one
 - Capacity will not get shrunk.

Adding an element to a specified position

```
package arrTest;
import java.util.ArrayList;

public class HelloWorld {

    public static void main(String[] args) {
        ArrayList<Integer> arr=new ArrayList<Integer>();
        arr.add(1); // add to the end
        arr.add(2); // add to the end
        arr.add(3); // add to the end
        arr.add(1,99);//add to the first position

        for(int i=0;i<arr.size();i++){
            System.out.println(arr.get(i));
        }//end of for

    }

}
```

1
99
2
3

Shifting will be done when
adding to position which is
filled

Adding to a specified position which does not exist

```
package arrTest;
import java.util.ArrayList;

public class arrListTest {

    public static void main(String[] args) {
        ArrayList<Integer> arr=new ArrayList<Integer>();
        arr.add(1); // add to the end
        arr.add(3,2); // add 2 to the third position

        for(int i=0;i<arr.size();i++){
            System.out.println(arr.get(i));
        }
    }
}
```

Exception in thread "main"
java.lang.**IndexOutOfBoundsException**
: Index: 3, Size: 1 at
java.base/java.util.ArrayList.rangeCheckForAdd(ArrayList.java:788)at
java.base/java.util.ArrayList.add(ArrayList.java:513)at
arrTest.main(arrTest.java:8)

The size of the array when we add 2 is 1. we are trying to add to 3rd position.

clear() and isEmpty()

```
package arrTest;
import java.util.ArrayList;

public class arrListTest {

    public static void main(String[] args) {
        ArrayList<Integer> arr=new ArrayList<Integer>();
        arr.add(1); // add to the end
        arr.add(2); // add to the end
        arr.add(3); // add to the end
        System.out.println("Before clear size is "+arr.size());
        arr.clear();
        System.out.println("After clear size is " +arr.size());
        if( arr.isEmpty() ){
            System.out.println("Arraylist is empty");
        }//end of if
    }
}
```

Before clear size is 3
After clear size is 0
Arraylist is empty

contains(Integer i)

```
package arrTest;
import java.util.ArrayList;

public class arrTest {

    public static void main(String[] args) {
        ArrayList<Integer> arr=new ArrayList<>();
        arr.add(1); // add to the end
        arr.add(2); // add to the end
        arr.add(3); // add to the end
        int num=2;
        if( arr.contains(num) ){
            System.out.println("Arraylist has " +num);
        }//end of if
    }
}
```

Arraylist has 2

//if you don't want exception

```
if (!arr.isEmpty() && arr.contains(2) ){
    System.out.println("Arraylist has 2");
}
```

indexOf(Integer i)

```
package arrTest;
import java.util.ArrayList;

public class arrTest {

    public static void main(String[] args) {
        ArrayList<Integer> arr=new ArrayList<>();
        arr.add(1); // add to the end
        arr.add(2); // add to the end
        arr.add(3); // add to the end
        int num=2;
        int searchedIndex = arr.indexOf(num);
        System.out.println(num+" is in " +searchedIndex);

    }

}
```

2 is in 1

**if the element not found what
will indexOf() return?**

-1

toString()

```
package arrTest;
import java.util.ArrayList;

public class arrTest {
    static ArrayList<String> languages;
    public static void main(String[] args) {
        languages = new ArrayList<>();
        // Add elements in the ArrayList
        languages.add("Java");
        languages.add("Python");
        languages.add("C");

        // convert ArrayList to String
        String list = languages.toString();
        System.out.println(list);
        System.out.println(languages);
    }
}
```

Returns a String which has all values of the arraylist

```
[Java, Python, C]
[Java, Python, C]
```

You can also just write the name of the ArrayList. It will still work.

remove(int index)

```
package arrTest;
import java.util.ArrayList;

public class arrTest {

    public static void main(String[] args) {
        ArrayList<Integer> arr=new ArrayList<>();
        arr.add(1); // add to the end
        arr.add(2); // add to the end
        arr.add(3); // add to the end
        int num=1;
        System.out.println("Before: pos 1 is "+ arr.get(num));
        // Remove the element at the 1st position
        int searchedIndex = arr.remove(num);
        System.out.println("After: pos 1 is " + arr.get(num));
        // the arraylist shifted
    }
}
```

removes the element at
that **index**

Before: pos 1 is 2
After: pos 1 is 3

remove(Integer element)

```
package arrTest;
import java.util.ArrayList;

public class arrTest {

    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<>();
        // Add elements in the ArrayList
        numbers.add(1);
        numbers.add(2);
        numbers.add(3);

        int num=2;
        numbers.remove(Integer.valueOf(num));
        //numbers.remove(Double.valueOf(num));
        //numbers.remove(Long.valueOf(num));
        System.out.println(numbers);
    }
}
```

removes the first occurrence
of the **element**

[1, 3]

remove(String element)

```
package arrTest;
import java.util.ArrayList;

public class arrTest {

    public static void main(String[] args) {
        ArrayList<String> languages = new ArrayList<>();
        // Add elements in the ArrayList
        languages.add("Java");
        languages.add("Python");
        languages.add("C");

        // convert ArrayList to String
        languages.remove("Java"); // remove Java element
        System.out.println(languages);

    }
}
```

removes the first occurrence
of the **element**

[Python, C]

Search and remove elements

```
package arrTest;
import java.util.ArrayList;
public class arrTest {
    public static void removeEls(ArrayList<Integer> arr, int num)
    {
        for(int i=0; i<arr.size(); i++) {
            if(arr.get(i) == num) {
                arr.remove(i);
            }
        }
    }

    public static void main(String[] args) {
        ArrayList<Integer> arrlist = new ArrayList<>();
        arrlist.add(1);arrlist.add(2);arrlist.add(3);
        arrlist.add(1);arrlist.add(1);arrlist.add(3);
        removeEls(arrlist,1);
        System.out.println(arrlist);
    }
}
```

Remove all occurrence
of an element in the
ArrayList

[2, 3, 1, 3]

Size of the ArrayList
changes when we
remove an element
from the ArrayList

Search and remove elements

```
package arrTest;
import java.util.ArrayList;
public class arrTest {
    public static void removeEls(ArrayList<Integer> arr, int num)
    {
        for(int i = arr.size() - 1; i>=0; i--) {
            if(arr.get(i) == num) {
                arr.remove(i);
            }
        }
    }

    public static void main(String[] args) {
        ArrayList<Integer> arrlist = new ArrayList<>();
        arrlist.add(1);arrlist.add(2);arrlist.add(3);
        arrlist.add(1);arrlist.add(1);arrlist.add(3);
        removeEls(arrlist,1);
        System.out.println(arrlist);
    }
}
```

Remove all occurrence of an element in the ArrayList

[2, 3, 3]

If you can traverse the list in reverse manner, you won't get effected by size reduction.
Can you implement the another solution but forward traverse?